ESPECIALIZACIÓN · ENTREGA 3

Mi primer programa en Python

Mi Negocio W

Calculadora básica del resultado mensual
de un pequeño emprendimiento

Estudiante	Laura Sardi
Modalidad	Actividad académica individual
Tema	Entrada, proceso y salida de información
Lenguaje	Python 3
Fecha	7 de noviembre de 2026

Propósito

Aplicar instrucciones de entrada, operaciones aritméticas y salida de resultados en un programa sencillo, legible y comentado.

Ejercicio académico inspirado en el contexto bancario de Banco W. El nombre Mi Negocio W se creó para esta actividad y no identifica una herramienta oficial del banco. Todos los datos de prueba son ficticios.

Aplicación publicada:
https://mi-negocio-w-laura-sardi.streamlit.app/
Código y despliegue documentados al final del PDF.

1. Problema propuesto y diseño

Una persona que administra un pequeño emprendimiento necesita identificar si las ventas de un mes cubren los costos de los productos o materiales vendidos y los demás gastos del negocio. Se propone un programa que solicite esos tres valores, calcule su diferencia y presente un resumen con ganancia, pérdida o equilibrio.

Etapa	Datos y operaciones
Entrada	ventas, costos y otros_gastos: enteros no negativos en pesos colombianos, del mismo mes. Ejemplo de entrada: 3000000, sin $ ni puntos ni comas.
Proceso	total_egresos = costos + otros_gastos. resultado = ventas - total_egresos.
Salida	Resumen de los tres montos, total de egresos, resultado y estado. Positivo: ganancia; negativo: pérdida; cero: equilibrio.

Reglas del ejercicio

Los costos y los otros gastos se registran por separado, sin repetir valores. El cálculo es simplificado: no incorpora impuestos, intereses ni créditos y no representa un estado financiero completo. Se aceptan ceros. Si hay un negativo o un dato no convertible a entero, se informa el problema y se debe ejecutar de nuevo.

Pseudocódigo del programa

```
INICIO
    DEFINIR pesos(valor): formatear entero como moneda
    ESCRIBIR título e instrucciones
    INTENTAR
        LEER ventas, costos, otros_gastos como enteros
        SI algún monto es negativo ENTONCES
            ESCRIBIR aviso de datos inválidos
        SINO
            total_egresos <- costos + otros_gastos
            resultado <- ventas - total_egresos
            SI resultado > 0: estado <- "GANANCIA"
            SINO SI resultado < 0: estado <- "PÉRDIDA"
            SINO: estado <- "EQUILIBRIO"
            ESCRIBIR resumen de montos, resultado y estado
        FIN SI
    SI FALLA LA CONVERSIÓN: ESCRIBIR aviso para reiniciar
    SI SE CIERRA O CANCELA LA ENTRADA: ESCRIBIR cierre
    FIN INTENTAR
FIN
```

2. Código fuente completo y comentado

Código íntegro en texto para poder copiarlo. La página siguiente contiene la captura real del mismo código. Conservar la sangría al guardarlo como mi_negocio_w.py.

```
"""Mi Negocio W: cálculo académico simplificado del resultado mensual."""


def pesos(valor):
    # Da formato al entero; los cálculos conservan los pesos originales.
    return "$" + format(valor, ",d").replace(",", ".")


print("MI NEGOCIO W - RESULTADO MENSUAL")
print("Ejercicio académico no oficial, inspirado en Banco W.")
print("Use COP enteros sin puntos ni comas y datos del mismo mes.")

try:
    # ENTRADA: input lee texto e int lo convierte a un número entero.
    ventas = int(input("Ventas del mes: "))
    costos = int(input("Costo de productos o materiales vendidos: "))
    otros_gastos = int(input("Otros gastos del negocio: "))

    # Los tres montos pueden ser cero, pero no pueden ser negativos.
    if ventas < 0 or costos < 0 or otros_gastos < 0:
        print("Datos inválidos: ningún monto puede ser negativo.")
    else:
        # PROCESO: sumar las salidas y restarlas de las ventas.
        total_egresos = costos + otros_gastos
        resultado = ventas - total_egresos

        # Clasificar el resultado sin confundir ventas con ganancia.
        if resultado > 0:
            estado = "GANANCIA"
        elif resultado < 0:
            estado = "PÉRDIDA"
        else:
            estado = "EQUILIBRIO"

        # SALIDA: presentar los datos, el cálculo y su interpretación.
        print("\nRESUMEN DEL MES (COP)")
        print("Ventas:", pesos(ventas))
        print("Costos:", pesos(costos))
        print("Otros gastos:", pesos(otros_gastos))
        print("Total de egresos:", pesos(total_egresos))
        print("Resultado:", pesos(resultado))
        print("Situación del negocio:", estado)
except ValueError:
    # Letras, vacíos o montos con punto/coma no se convierten a entero.
    print("Entrada inválida. Reinicie y use pesos enteros sin separadores.")
except (EOFError, KeyboardInterrupt):
    print("\nPrograma finalizado: entrada cerrada o cancelada.")
```

3. Captura del código fuente

Captura real de pantalla del código completo abierto en un editor de texto. El contenido coincide con el archivo Python ejecutado en las pruebas.

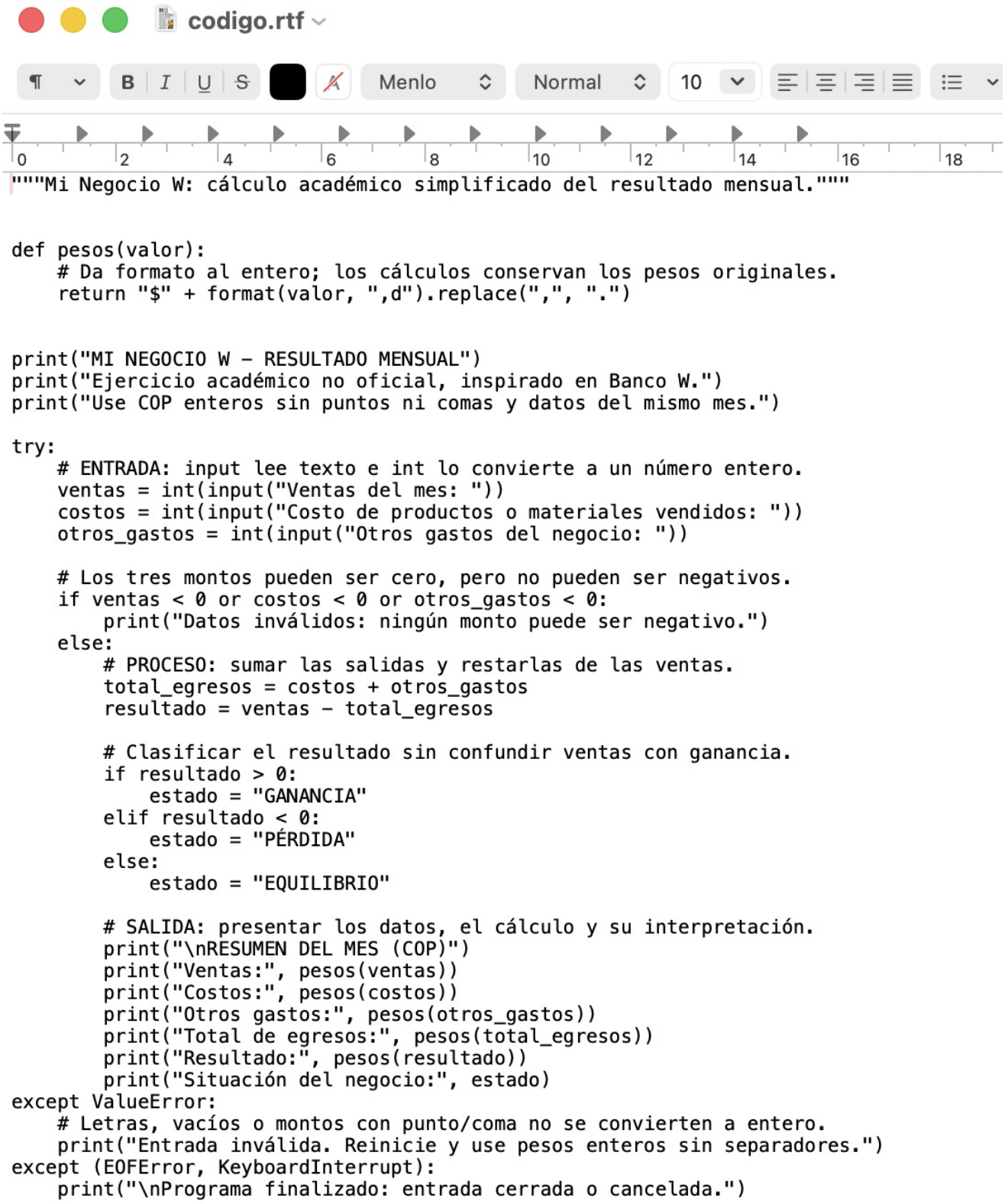


Figura 1. Se observan las entradas, el cálculo, los comentarios y la salida.

4. Funcionamiento y ejecución

Cómo se organiza el programa

Componente	Explicación
Entrada y tipos	input() recibe texto; int() lo convierte a entero [1]. ventas, costos y otros_gastos conservan valores en COP enteros. Las comparaciones generan valores booleanos.
Proceso secuencial	Después de leer y validar, se suman costos y otros gastos. Luego se resta ese total a las ventas. El orden permite usar datos ya definidos.
Interpretación	if, elif y else asignan a estado una cadena con la categoría. Una pérdida es un resultado negativo; no significa que los datos de entrada deban ser negativos.
Salida	print() muestra etiquetas y valores [1]. pesos() convierte el entero a una cadena con $ y puntos de miles [2]. El formato no altera los cálculos.
Manejo de errores	try/except captura ValueError si falla la conversión [3]. EOFError y KeyboardInterrupt permiten cerrar la entrada o cancelar sin mostrar un rastreo técnico.
Buenas prácticas	Los nombres expresan qué guarda cada variable; los comentarios identifican entrada, proceso y salida; la sangría separa los bloques. No se requieren ciclos para una única consulta mensual.

Instrucciones de ejecución

1. Tener Python 3 disponible y abrir IDLE. Elegir File > New File, copiar el código completo de la página 3 y guardarlo como **mi_negocio_w.py**.

2. Elegir Run > Run Module (F5). Ingresar los tres montos cuando el programa los solicite y pulsar Enter después de cada uno. Usar datos del mismo mes, en pesos enteros, sin separadores.

3. Leer el resumen. Para analizar otro caso o corregir una entrada inválida, volver a ejecutar. También se puede iniciar desde su carpeta con **python3 mi_negocio_w.py** en macOS/Linux o **py mi_negocio_w.py** en Windows.

El programa usa únicamente funciones integradas de Python. No necesita importar librerías, instalar paquetes con pip, usar Excel ni conectarse a internet. Fue probado con Python 3.12.14.

5. Ejecución exitosa: ganancia

Captura real del visor local del registro de una ejecución en pseudoterminal. Las entradas fueron enviadas al programa y reproducidas por la terminal; se muestran junto a sus preguntas.

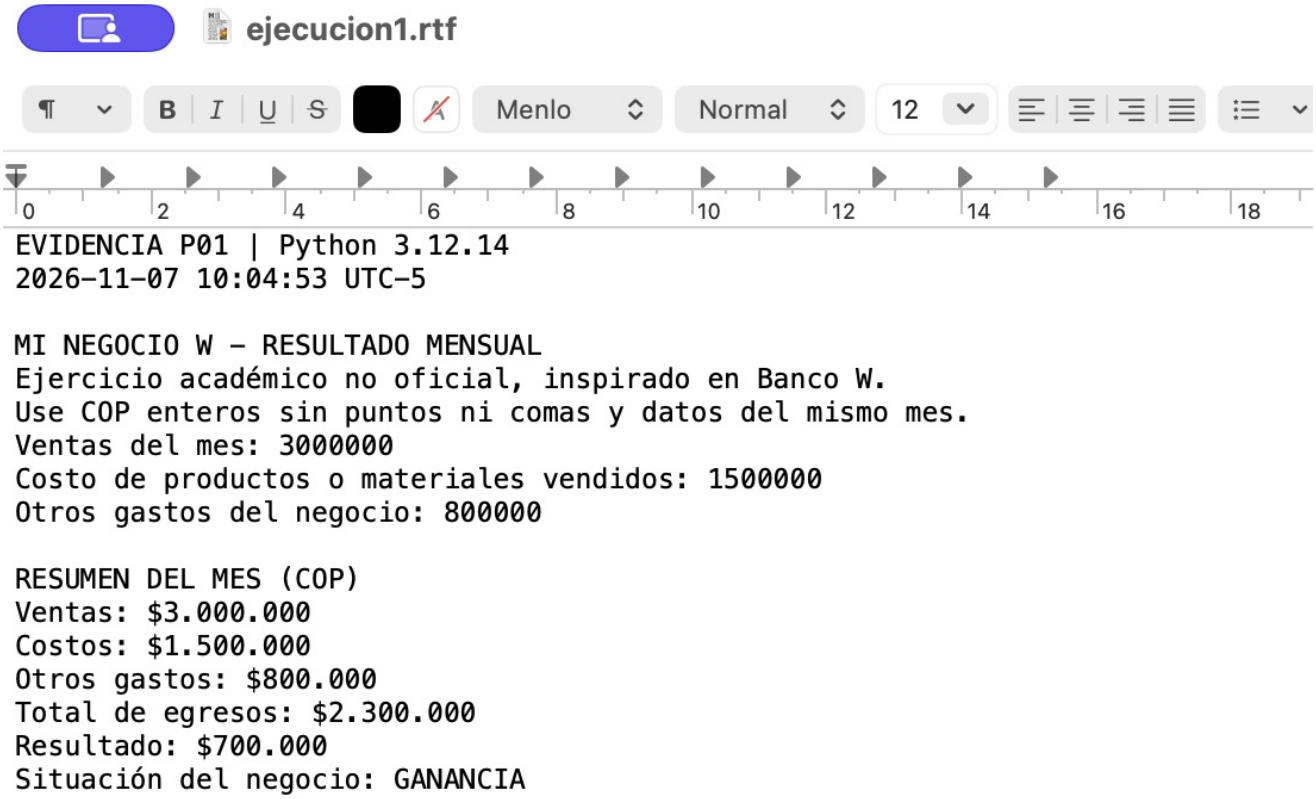


Figura 2. Ventas de $3.000.000, costos de $1.500.000 y otros gastos de $800.000 producen una ganancia de $700.000.

Comprobación manual

```
Total de egresos = 1.500.000 + 800.000 = 2.300.000 COP
Resultado        = 3.000.000 - 2.300.000 = 700.000 COP
700.000 > 0      => GANANCIA
```

El resumen conserva los valores de entrada y diferencia las ventas del resultado. Esto evita interpretar todo el dinero vendido como ganancia.

5.1. Ejecuciones: pérdida y equilibrio

Captura real del registro de dos ejecuciones independientes. Cada una incluye tanto los datos ingresados como la salida completa. La línea horizontal separa las sesiones.

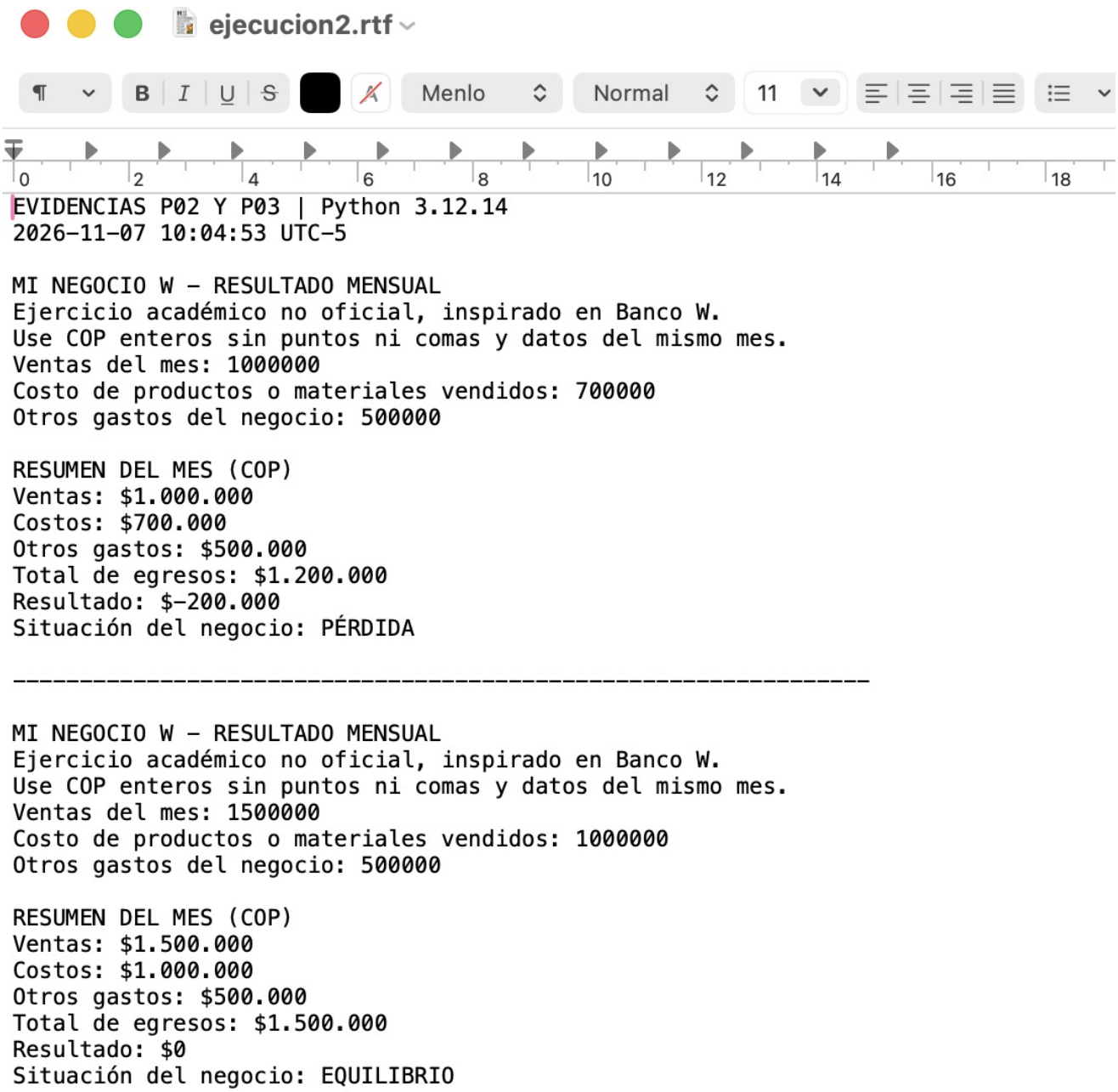


Figura 3. Primer caso: $1.000.000 - $700.000 - $500.000 = **-$200.000**, pérdida. Segundo caso: $1.500.000 - $1.000.000 - $500.000 = **$0**, equilibrio.

En la salida, $-200.000 expresa el resultado negativo de 200.000 COP. Las capturas proceden del programa ejecutado; el editor solo permite ver su registro, sin reconstruir las respuestas.

6. Pruebas, reflexión y fuentes

Se realizaron 15 pruebas el 7 de noviembre de 2026, a las 10:04:53 (Colombia, UTC-5), con Python 3.12.14. Todas terminaron normalmente y sin salida de errores técnicos.

Casos	Entrada o condición	Esperado y obtenido
P01-P03	Ganancia, pérdida y equilibrio	700.000; -200.000; 0 COP. Categorías correctas.
P04	0, 0, 0	Resultado 0: equilibrio.
P05-P06	1001 o 999; costos 500; gastos 500	Ganancia de 1 o pérdida de 1 COP.
P07-P09	Negativo en cada uno de los tres campos	Aviso de datos inválidos; no presenta resumen.
P10-P13	hola; vacío; 3.5; 1.000	Aviso de formato inválido; no presenta resumen.
P14	Ventas “ +1000 ”; costos 400; gastos 100	Resultado 500 COP: ganancia.
P15	Cierre de entrada sin datos	Mensaje de cierre, sin fallo técnico.

Las tres primeras pruebas se contrastaron con las restas mostradas en las evidencias. Los casos de ±1 verifican el límite de clasificación; los ceros y los datos inválidos verifican que el programa no confunda un valor permitido con un error.

Reflexión personal

Al desarrollar Mi Negocio W comprendí cómo se conectan las entradas, el procesamiento y las salidas de un programa. Aprendí que input recibe texto y que debo convertirlo a un tipo numérico antes de hacer operaciones. También entendí que vender no equivale a ganar: primero se deben descontar los costos y los otros gastos. Una dificultad fue mantener separados estos valores para no contar dos veces el mismo concepto; la resolví definiendo con claridad cada entrada. Otra fue manejar letras o montos mal escritos, por lo que incorporé avisos que explican cómo corregirlos. Probar ganancia, pérdida, equilibrio y valores cercanos a cero me ayudó a revisar tanto el cálculo como su interpretación. Este ejercicio me mostró que un programa pequeño puede ser útil cuando sus instrucciones son claras y sus resultados se comprueban. Como mejora futura, permitiría repetir consultas sin reiniciar y guardar un historial mensual.

Fuentes consultadas

[1] Python Software Foundation. *Funciones incorporadas.* input, int y print.
https://docs.python.org/es/3/library/functions.html
[2] Python Software Foundation. *Entrada y salida.* Presentación y formato.
https://docs.python.org/es/3/tutorial/inputoutput.html
[3] Python Software Foundation. *Errores y excepciones.*
https://docs.python.org/es/3/tutorial/errors.html
Consulta: 7 de noviembre de 2026. Enunciado académico proporcionado por la estudiante.

7. Proyecto en GitHub y Streamlit

El proyecto Python está publicado en un repositorio público de GitHub y desplegado en Streamlit Community Cloud. Se puede usar desde un navegador sin instalar Python.

Usar la calculadora

https://mi-negocio-w-laura-sardi.streamlit.app/

Consultar el repositorio

https://github.com/Laurasardig/mi-negocio-w

Cómo comprobarla: elegir un ejemplo o ingresar los tres montos; pulsar **Calcular resultado**; revisar el resumen. Los desplegables permiten consultar el código Python de consola y el de Streamlit.

Relación entre las dos versiones

El código de consola de este PDF conserva sus entradas con input y sus salidas con print. En la versión web, **app.py** reemplaza esas interacciones por un formulario y componentes de Streamlit; **calculos.py** aplica la misma suma, resta y clasificación. Los tres ejemplos se comprobaron también en la aplicación publicada.

La consola no necesita paquetes externos. Para ejecutar la versión web localmente se instala la dependencia de requirements.txt y se inicia **streamlit run app.py**. El despliegue usa Python 3.12 y Streamlit 1.67.0. La versión web procesa los datos en el servidor; se utilizan valores ficticios y no se crea un historial.

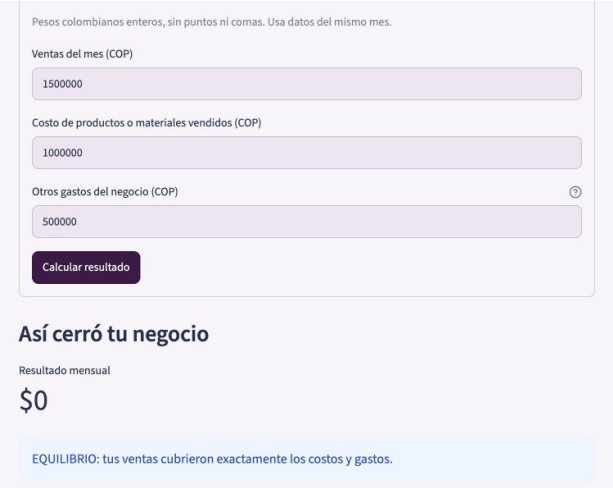


Figura 4. Resultado verificado en la aplicación pública de Streamlit.